Section 6 — Serving & Real-Time Systems

The latency budget, precompute-vs-live, model freshness, efficiency — plus a dedicated treatment of ByteDance Monolith. Companion to Sections 1–5.

Contents

Sections 1–5 built the model and how to evaluate it. This section is about *running it in production*: the constraints that shape every architectural choice, and the systems tricks that make a heavy model shippable. The architecture itself (funnel, Two-Tower, ANN, re-ranking) is assumed from Sections 1–4; here we cover only what's serving-specific.

6.1 — The latency budget: why the whole funnel exists

Serving has one hard constraint the offline world doesn't: a **per-request latency budget** — the time to return a feed before the user feels lag. The two sources give slightly different figures for the same idea:

source	budget
Alex Xu (p.130)	≤ 200 ms
Hello Interview (p.1)	~250 ms

That budget is the *reason for the multi-stage funnel*. You cannot run the heavy ranker — a transformer pass per candidate — across billions of videos in 200 ms. So the funnel is a **latency optimization, not only a quality one**: cheap models cut billions → thousands, progressively heavier models refine thousands → hundreds → dozens, and the expensive model only ever scores the ~hundreds that survive.

Interview framing: "the multi-stage design isn't accuracy-first — the latency budget makes scoring everything with the heavy model impossible, so I narrow cheaply first and spend the expensive compute only on survivors."

6.2 — Precompute vs compute-live: the core serving trade-off

The Instagram Explore leitmotif: *clever use of caching and precomputation lets you run heavier models at every stage without blowing the budget.* The skill is knowing what to precompute offline vs compute live, and the dividing line is one principle:

> **Anything that doesn't depend on the live request can be precomputed. Anything that needs the freshest request-time signal must be computed online.**

Precomputed offline (often during off-peak hours)

- **Item (video) embeddings** — a video's Two-Tower vector depends only on the video, not the user, so the item tower runs in a daily offline pipeline and the embeddings are indexed in the ANN service. This is *why* the Two-Tower can't use user–item cross features: using them would make the item embedding user-dependent and destroy its cacheability.
- **Candidate-generation results**, and for some users even **full second-stage recommendations**, are precomputed during off-peak hours. The heavy MTML model is expensive at peak, so shifting some of that computation to idle hours keeps recommendations *available* for every user under peak load — Instagram's stated reason is availability, not just cost. It's load-shifting, not a permanent cache: precomputed recommendations can go stale and still get refreshed by live ranking.

Computed live (per request)

- **The user embedding** — computed on the fly from the *freshest* user-side features (what they watched 10 seconds ago matters), then used to query the ANN index. You precompute the haystack (item embeddings), not the needle (the user's current state).

The cache-miss fallback: embeddings are served from caches (user-side and item-side); on a cache miss the system recomputes that one embedding via the tower. Caching is best-effort — a hit is cheap, a miss falls back to live computation rather than failing.

The pattern: precompute everything request-independent during off-peak, compute only the request-dependent user state live, cache aggressively with recompute-on-miss. That's what lets each funnel stage use a heavier model than its raw latency would otherwise allow.

6.3 — Model freshness: keeping a live model current

A recommendation model goes stale fast — trends shift hourly, new videos arrive constantly, and last week's model mis-ranks today's content. (The Monolith paper calls this *concept drift*: the underlying distribution of user behavior is non-stationary.) Source-backed mechanisms keep it fresh:

- **Continual / online training (Instagram). Both ranking stages** are fine-tuned hourly — the blog says "for both stages," meaning the first-stage Two-Tower light ranker *and* the second-stage heavy MTML ranker are re-trained (fine-tuned) every hour as new data arrives. This is *why* neural nets were chosen for both stages: fast adaptability. Note the cadence differs from retrieval, where item embeddings refresh on the slower daily offline pipeline (6.2).
- **Efficient online updates, e.g. ByteDance's Monolith** (Hello Interview names it). Covered in full in Section 7.
- **Cold-start fine-tuning (Alex Xu).** When a new video has gathered a little interaction data, the Two-Tower is fine-tuned to fold it in. (Full cold-start is a Section 8 topic; here it's just the freshness angle.)

How does the hourly fine-tuning actually work?

Don't start over — *continue*. The model already has trained weights from its last version. Fine-tuning **warm-starts** from those weights and runs a few gradient steps on *only the newest data* (the last hour's interactions), nudging the weights to absorb recent behavior. The cycle: **(1)** collect the last hour's interactions as fresh (features, label) examples; **(2)** load the currently-serving model as the starting point — not random init; **(3)** run a few gradient steps on that small recent batch using the same loss (for the heavy ranker, $L = L_{engage} + L_{aux} + L_{position}$); **(4)** validate and hot-swap it in, then repeat next hour from the new weights. Warm-starting is what makes it cheap: training from scratch re-learns everything from all history (hours/days); fine-tuning re-learns only the *delta* on top of an already-good model (minutes). What actually moves is mostly the recency-sensitive parameters — the embedding tables for new/trending IDs and the upper layers — while the deep interaction machinery learned over months barely budges in any one hour.

---

*The tension: freshness vs cost, and freshness vs stability. Fine-tuning tracks trends fast but risks **drift / catastrophic forgetting** — if each hour sees only the last hour's data, the model can over-fit recent noise and forget stable long-term preferences. Mitigations: mix older data into each batch, keep the learning rate small, and periodically do a full retrain from scratch to reset accumulated drift. The real pattern is hourly fine-tune for freshness + occasional full retrain for stability.*

6.4 — Hashing & embedding tables

Hashing runs quietly under every embedding in this system (the Two-Tower vectors, the heavy ranker's categorical embeddings, the caches). Understanding it is the prerequisite for the Monolith section, because Monolith's headline feature is a *fix* for the standard hashing approach.

The problem: unbounded IDs, a fixed table

Every categorical ID — user, video, creator — needs an embedding. But there are *billions* of videos and the set grows every second; you can't allocate one row per possible ID in advance. The standard fix is the **hashing trick**: fix the table size, then map each ID into it.

> **embedding_row = hash(video_id) mod TABLE_SIZE**

What does "sparse categorical feature" mean here?

"Categorical" = a feature whose value is one of a huge discrete set (a video ID is one of billions, not a number you do math on). "Sparse" = for any one example, almost all possible values are absent — a single impression involves one video out of billions, so the one-hot representation is a vector that is all zeros except one entry. Embeddings exist precisely to turn these sparse one-hot IDs into small dense vectors. The hashing trick is how the system decides *which* dense vector a given ID points to.

WORKED EXAMPLE — THE HASHING TRICK AND A FORCED COLLISION (VERIFIED)

Tiny table of 5 rows (real tables are ~10M); map 7 video IDs via `hash(id) mod 5`:

```
hash(vid_A) mod 5 -> row 3        row 0: vid_C
hash(vid_B) mod 5 -> row 4        row 1: vid_D
hash(vid_C) mod 5 -> row 0        row 2: vid_E
hash(vid_D) mod 5 -> row 1        row 3: vid_A, vid_F   <-- COLLISION
hash(vid_E) mod 5 -> row 2        row 4: vid_B, vid_G   <-- COLLISION
hash(vid_F) mod 5 -> row 3
hash(vid_G) mod 5 -> row 4
```

7 IDs into 5 rows means `vid_A` and `vid_F` are **forced to share one embedding**. The model then cannot tell them apart — if A is cooking and F is football, they're blended into one averaged vector, and every downstream computation (Two-Tower similarity, the heavy ranker, diversity) inherits that corruption. With billions of IDs in a 10M table, collisions are guaranteed (pure pigeonhole: more IDs than rows).

This is the exact failure mode Monolith was built to eliminate — see the dedicated Monolith section that follows the serving topics.

6.5 — Efficiency: fitting the model in the budget

Two source-named techniques (Hello Interview) make a heavy model shippable.

Quantization

Store and run the model at lower numerical precision (FP32 → FP16 → INT8): smaller memory footprint, faster inference, minimal accuracy loss.

The mechanics below are standard ML, supplied to develop what the source only names. The arithmetic is verified.

precision	bytes / param	1B-param model	relative size
FP32	4	4.0 GB	baseline
FP16	2	2.0 GB	½
INT8	1	1.0 GB	¼

WORKED EXAMPLE — INT8 MAPPING (VERIFIED)

INT8 maps a weight's float range onto 256 integer levels (−128…127). For weights spanning [−6, 6] over 256 levels, the step is 12 / 255 ≈ **0.047** per level. A weight of 2.37 maps to integer level round((2.37−(−6))/0.047) = **178**, which dequantizes back to 178 × 0.047 − 6 = **2.376** — a rounding error of just 0.006. That tiny per-weight error is the price for **4× less memory and ~2–4× faster inference**.

Why it matters for serving: smaller models fit in GPU memory, load faster, and run more inferences per second — directly buying back latency budget and throughput.

GPU / TPU serving

Recommendation models are matrix-heavy — exactly what GPUs/TPUs accelerate. Serving on them (vs CPU) raises **throughput** (requests per second), letting the system survive peak load within the same latency budget.

How Section 6 connects back to Section 5

Section 5 listed latency, throughput, and resource utilization as *system-health guardrails* — constraints a model must satisfy to ship. Section 6 is *how you satisfy them*: the funnel keeps the heavy model off billions of candidates (latency); precompute + caching minimize per-request work (latency + utilization); freshness keeps the model accurate without full retrains (cost); Monolith's collisionless tables + minute-level sync keep it fresh in real time (quality under drift); quantization + accelerators squeeze it into budget (latency + throughput + utilization). Serving is where the evaluation guardrails become engineering decisions.

Section 7 — ByteDance Monolith (real-time recommendation)

This section is faithful to the original paper: "Monolith: Real Time Recommendation System With Collisionless Embedding Table," Liu et al., ByteDance, ORSUM@ACM RecSys 2022 (arXiv 2209.07663). Every claim, mechanism, and number below is from that paper; nothing is invented. Hello Interview names Monolith as one option for efficient online updates — this section is the deep dive into what it actually does.

7.1 — Why Monolith exists: two problems with general frameworks

The paper's thesis: general-purpose frameworks like TensorFlow and PyTorch fall short for recommendation for two reasons rooted in the *nature of the data*:

1. **Features are sparse, categorical, and dynamically changing** — and the number of users/items is orders of magnitude larger than, say, the vocabulary of a language model, so the embedding table cannot fit a fixed dense variable and must *grow over time* as new IDs are admitted.
2. **The data distribution is non-stationary (concept drift)** — a user's interest can shift "every next minute," so a model trained on yesterday's batch and frozen at serving cannot reflect the latest interest. The batch-train-then-serve split that general frameworks assume *structurally prevents* real-time adaptation.

Monolith's two contributions map exactly onto these two problems: a **collisionless embedding table** (for problem 1) and an **online training architecture** (for problem 2).

7.2 — The collisionless embedding table (problem 1)

What's wrong with the standard fix. The paper notes that many systems use low-collision hashing to bound the table and allow growth — but this "relies on an over-idealistic assumption that IDs... are distributed evenly in frequency, and collisions are harmless." In reality IDs are **long-tailed** (a few users/items occur millions of times, most appear rarely), and as the table grows, collisions increase and *degrade model quality*. The paper's first design principle, quoted: *"avoid cramping information from different IDs into the same fixed-size embedding."*

> **FROM THE PAPER — COLLISIONS MEASURABLY HURT QUALITY**
>
> On MovieLens ml-25m, they hashed IDs into a smaller space to force collisions and compared. Even at modest collision rates the collisionless table won on AUC from the first epoch and converged higher; on the production model the collisionless table stayed more robust over time as concept drift shifted the data.

	User IDs	Movie IDs
Before hashing	162,541	59,047
After hashing	149,970	57,361
Collision rate	**7.73%**	**2.86%**

Paper Table 1. Even ~3–8% collisions measurably lowered AUC — the empirical justification for going collisionless.

How the collisionless table works: Cuckoo hashing

The actual mechanism (paper §2.1) is a **Cuckoo HashMap**, not a naively growing dictionary. It keeps **two tables T_0 and T_1 with two different hash functions h_0 and h_1**; each element lives in exactly one of them.

HOW A CUCKOO INSERT AVOIDS COLLISIONS

To insert element A: try slot $h_0(A)$ in T_0. If it's empty, place A there — done. If it's occupied by B, **evict B**, put A in, and re-insert B into T_1 at $h_1(B)$. If *that* slot is occupied, evict its occupant and cascade, until everything stabilizes — or a cycle is detected, which triggers a rehash. The "cuckoo" name is the bird that pushes others out of the nest. The key property: a new key can always be inserted **without permanently colliding into an existing one** — someone gets relocated, nobody gets blurred.

Performance guarantees from the paper: worst-case **O(1)** lookups and deletions, amortized **O(1)** insertions.

Bounding memory: two eviction heuristics

Unique slots for every ID would grow memory without bound, so Monolith filters and expires IDs. The paper's two justifying observations and the resulting heuristics:

- **Frequency filtering (on admission).** Observation: IDs are long-tailed, and rare IDs (seen only a handful of times) are *underfit* — too little data to learn a good embedding — so they "are not likely to affect the result." Mechanism: filter IDs *before* admitting them as keys, using (a) an occurrence-count threshold (a tunable hyperparameter per model) and (b) a probabilistic filter to further cut memory.
- **Expiry (by inactivity).** Observation: stale IDs from the distant past (an inactive user, an out-of-date video) "seldom contribute to the current model." Mechanism: IDs are timed and *expire* after a tunable inactive period — different per table, so features with different sensitivity to history get different expiry.

7.3 — The online training architecture (problem 2)

After a model is deployed, *training does not stop.* The paper splits training into two stages:

- **Batch training** — the ordinary loop over historical data (used when modifying architecture and retraining from scratch). Notably they train the dataset for only *one pass*.
- **Online training** — after deployment, a worker consumes *real-time data on the fly* and updates the training parameter server (PS), which periodically syncs to the serving PS, taking effect on users immediately. This is what closes the feedback loop in near-real-time.

What is a "parameter server (PS)"?

Monolith follows TensorFlow's **Worker–Parameter-Server** architecture. **Workers** do the computation (forward/backward passes); **Parameter Servers** store the model's parameters and apply gradient updates. Parameters split into two kinds: **dense** (the neural-net weights) and **sparse** (the embedding tables — which dominate the model's size). This dense/sparse split matters for the sync trick below.

The streaming engine

The data plumbing (paper §2.2.1, Fig 4–5): a Kafka queue logs *user actions* (click, like, …) and another logs *features*. An Apache Flink streaming job — the **online Joiner** — concatenates features with their labels (from user actions) to produce training examples, written to a third Kafka queue consumed by both online and batch training.

Why is joining features with labels non-trivial? (two real problems the paper names)

(1) Out-of-order arrival. Action logs and feature logs stream in without time-order guarantees, so each request gets a *unique key* so the action and its features can be paired correctly. **(2) Action lag.** A user might act days after seeing an item (e.g. a delayed purchase). Holding all features in memory that long won't fit, so features waiting beyond a time window spill to an on-disk key-value store; when an action arrives, the joiner checks the in-memory cache first, then the KV store on a miss. Also, because negatives vastly outnumber positives, the joiner does **negative sampling**, then applies **log-odds correction** at serving so the model remains an unbiased estimator despite the resampling.

Parameter synchronization — the clever part

The challenge: production models are multi-terabyte, can't stop serving to update, and transferring the whole model would crush network and memory. Monolith exploits three facts (paper §2.2.3):

1. Sparse parameters (embeddings) dominate model size.
2. In a short window, only a *small subset* of IDs gets trained, so only their embeddings change.
3. Dense weights move *much slower* than sparse embeddings (momentum accumulation over huge data magnifies dense stability, while only a few embeddings update per batch).

So Monolith syncs **sparse embeddings frequently (minute-level)** — pushing only the "touched-keys" set (IDs trained since last sync) — and **dense weights infrequently (daily)**. Serving with a slightly stale dense part alongside fresh sparse parts is tolerable and causes "no conspicuous loss."

WORKED EXAMPLE — WHY INCREMENTAL SYNC IS CHEAP (THE PAPER'S OWN NUMBERS, VERIFIED)

Suppose 100,000 IDs get touched in a minute and the embedding dimension is 1024 (4 bytes/value):

```
data per sync = 100,000 IDs x 1024 dims x 4 bytes
              = 409,600,000 bytes
              ≈ 400 MB per minute
```

That's a small, network-friendly incremental push — versus transferring a multi-terabyte model. Pushing *only touched sparse keys* at minute-level is what makes real-time sync feasible. Dense parameters sync daily, scheduled at lowest-traffic times (e.g. midnight).

7.4 — The headline trade-off: reliability for real-time

The paper's third contribution is proving that **system reliability can be traded off for real-time learning** — and that the trade is worth it. Two findings:

- **More frequent sync → better quality.** On the Criteo dataset, online serving AUC improved monotonically as the sync interval shrank (5 hr → 1 hr → 30 min). A live A/B on a production ads model showed online training beating batch training by ~14–18% AUC improvement across 7 days.
- **Snapshots can be rare.** Counter-intuitively, they enlarged the fault-tolerance snapshot interval to **once per day** and saw nearly no quality loss. Their reasoning: with parameters sharded across 1000 PS at a 0.01% daily failure rate, a failure loses one day of updates from a tiny fraction (~0.01%) of users' sparse features — negligible, and dense variables move slowly anyway. So they "radically lowered snapshot frequency" and saved compute.

The Monolith lesson in one line: sync the fast-moving sparse embeddings as often as possible (minute-level), sync slow dense weights rarely (daily), snapshot for recovery even more rarely (daily) — trading a little reliability for near-real-time freshness, which the experiments prove is a net win.
